```
[OUTPUT]
    Name opensearch
    Match **
    Host opensearch-node1
    Port 9200
    Index ho11y
    Suppress_Type_Name on
```

The destination configuration—in our case, the OpenSearch plug-in as the backend (downstream) for the log items

Here, we define that we, indeed, take any log item (we could be more selective here regarding which ones we're routing to OpenSearch, but for now, we use all of them).

The name of the index we want to create in OpenSearch

Tells the OpenSearch node that we don't have to set a type (since we don't care, in the context of this setup)

The hostname (which could also be a raw IP address) of the OpenSearch node

Phew, that was a lot of configuration! Now, it's time to launch the pipeline and generate some log lines. First off, we need to launch the Fluent Bit–OpenSearch combo:

```
docker-compose -f docker-compose.yaml up
```

In a different terminal session, launch a simple load generator (in Bash shells) like so:

```
while true ; do \
  curl -s -o /dev/null -w "%{http_code}"
  ➥ localhost:8765 ; \
  sleep 1 ; \
done
```

Uses curl to invoke ho11y and generate log lines, discarding the output and only showing the HTTP status code returned

Pauses for 1 second and then repeats from the top

Now, you can navigate to http://localhost:5601/. You should see the Welcome page of OpenSearch Dashboards (in the ELK stack—Elasticsearch, Logstash, and Kibana—the equivalent component would be Kibana). Click on the burger icon in the upper-left corner (the three lines), and select Stack Management followed by Index Patterns; you should see something like what is shown in figure 4.2.

Next, click on Create Index Pattern, and in the input box that's labeled Index Pattern Name, enter `ho11y` and then click Next Step. Then, in the Time Field dropdown list, select `@timestamp` followed by clicking the Create Index Pattern button. The result you should see is something like what is depicted in figure 4.3.

Now, head back to the OpenSearch Dashboards home page, click on the burger icon in the upper-left corner (the three lines) and then click Discover. You should then see something akin to what is shown in figure 4.4.

You now have a functioning logging pipeline based on Fluent Bit as a log router at your disposal. From this point on, feel free to drill into log lines or use the Dashboards Query Language (DQL; https://opensearch.org/docs/latest/dashboards/dql), a domain-specific query language, to find data in OpenSearch indexes, to query for specific terms in the `ho11y` logs.